

Sequence Diagram

```
participant User
participant Student
participant API
participant TrackingService
participant DB
```

```
User->>Student: Login to System
Student->>API: Request Bus Location
API->>TrackingService: fetchBusLocation()
```

```
TrackingService->>DB: Get GPS Data
```

```
alt Bus Data Available
    DB-->>TrackingService: Bus Coordinates
    TrackingService-->>API: Live Bus Status
    API-->>Student: Display Bus Location
else Bus Offline
    TrackingService-->>API: No Data Available
    API-->>Student: Error Response
end
```

Architectural diagram

flowchart TD

User --> Frontend[React App / Mobile App]

Frontend --> API[REST API / Azure API Management]

API --> AppService[Application Service]

AppService --> DB[(MySQL / Azure SQL Database)]

AppService --> GPS[GPS Tracking Device]

GPS --> TrackingService[Bus Tracking Service]

TrackingService --> Queue[Message Queue / Service Bus]

Queue --> Worker[Background Worker]

Worker --> DB

AppService --> Monitor[Azure Monitor / Dashboard]

ER Diagram

erDiagram

```
STUDENT {  
    int student_id PK  
    string student_name  
    string department  
    string phone_number  
}
```

```
BUS {  
    int bus_id PK  
    string bus_number  
    string route_name  
    string driver_name  
}
```

```
ROUTE {  
    int route_id PK  
    string source  
    string destination  
    string stop_name  
}
```

```
TRACKING {  
    int tracking_id PK  
    int bus_id FK  
    string current_location  
    datetime updated_time  
}
```

```
STUDENT ||--o{ BUS : travels  
BUS ||--o{ ROUTE : follows  
BUS ||--o{ TRACKING : updates
```

Case Diagram

```
@startuml
left to right direction

actor Admin
actor Student
actor Driver
actor "GPS Device" as GPS

rectangle "College Bus Tracking System" {

Admin --> (Add Bus)
Admin --> (Update Bus Details)
Admin --> (Delete Bus)
Admin --> (View Bus Reports)

Student --> (Track Bus)
Student --> (View Bus Route)
Student --> (Receive Notifications)|

Driver --> (Update Bus Status)
Driver --> (Share Live Location)

GPS --> (Send GPS Coordinates)

(Track Bus) --> (Fetch Live Location)
(Fetch Live Location) --> (Display Current Location)

(Display Current Location) --> (Show Estimated Arrival Time)

Admin --> (Monitor Bus Movement)
}
@enduml
```

Class Diagram

```
class Student {  
    +int studentId  
    +string studentName  
    +string department  
    +string phoneNumber  
}
```

```
class Bus {  
    +int busId  
    +string busNumber  
    +string driverName  
    +string routeName  
}
```

```
class Route {  
    +int routeId  
    +string source  
    +string destination  
    +string stopName  
}
```

```
class Tracking {  
    +int trackingId  
    +string currentLocation  
    +datetime updatedAt  
}
```

Student --> Bus

Bus --> Route

Bus --> Tracking

